

Báo cáo Thực nghiệm: LeNet-5 trên Tập dữ liệu CIFAR-10

Assignment 4 · Kiến trúc Cổ điển Đối diện Thách thức Thị giác · Scratch NumPy · TensorFlow/Keras · PyTorch

Báo cáo chuyên sâu tổng hợp kết quả cho notebook `07_cifar10_lenet.ipynb`. Notebook cơ sở `03_cifar10.ipynb` với mô hình `SmallCNN` **được duy trì nguyên vẹn** làm hệ quy chiếu đối sánh khách quan.

Mọi số liệu thực nghiệm được kết xuất trực tiếp từ các file kết quả thực thi cục bộ `results/07_cifar10_lenet.json` và `results/03_cifar10.json`.

1. Động lực Thử nghiệm LeNet-5 trên CIFAR-10

1.1 Câu hỏi Nghiên cứu Trọng tâm

Mạng LeNet-5 (LeCun et al., 1998) được thai nghén chuyên biệt cho ảnh **thang xám 32×32** chứa ký tự chữ số đơn sắc được căn giữa. Trái lại, CIFAR-10 bao gồm ảnh **màu ba kênh RGB 32×32** ghi lại 10 lớp đối tượng tự nhiên đa dạng: máy bay, xe hơi, chim chóc, mèo, hươu, chó, ếch, ngựa, tàu thủy, xe tải.

Kích thước hình học không gian đầu vào hoàn toàn trùng khớp (32×32), song độ phức tạp ngữ nghĩa thị giác có sự phân hóa sâu sắc.

Thực nghiệm này được thiết kế nhằm cô lập và kiểm chứng giả thuyết:

Một kiến trúc kinh điển từ thập niên 1990 sẽ biểu hiện ra sao khi đối mặt với bài toán nhận diện thực thể tự nhiên vượt ngoài phạm vi thiết kế ban đầu?

Báo cáo `08_lenet_mnist_report.pdf` đã chỉ ra rằng trên MNIST, LeNet-5 đạt hiệu năng tương đương với baseline. Báo cáo này làm rõ liệu nhận định trên có còn giữ nguyên giá trị khi độ phức tạp phân phối dữ liệu tăng vọt.

1.2 Kiến trúc Mạng tổng quát

Ảnh đầu vào $3 \times 32 \times 32$ (RGB)			
	↓		
C1	Conv 6 @ 5×5 , pad=0	→ $6 \times 28 \times 28$	Bản đồ đặc trưng không gian
	↓ ReLU		
S2	MaxPool 2×2	→ $6 \times 14 \times 14$	Lớp gộp subsample
	↓		
C3	Conv 16 @ 5×5 , pad=0	→ $16 \times 10 \times 10$	Bản đồ đặc trưng bậc hai
	↓ ReLU		
S4	MaxPool 2×2	→ $16 \times 5 \times 5$	Lớp gộp subsample
	↓ Flatten	→ 400 chiều	
F5	Dense 400 → 120	↓ ReLU	
F6	Dense 120 → 84	↓ ReLU	
out	Dense 84 → 10	(Điểm số Logits ngõ ra)	

Khác biệt cấu trúc so với biến thể MNIST: Tầng C1 áp dụng `pad=0` (tích chập không đệm - *valid*). Do ảnh CIFAR-10 vốn đã có kích thước chuẩn 32×32 (đúng kích thước thiết kế gốc của Yann LeCun), phép tích chập không đệm tự nhiên chuyển đổi ảnh về 28×28 , khớp hoàn toàn với cấu trúc LeNet-5 nguyên thủy.

1.3 Phân bổ Trọng số Tham số

Tầng mạng	Số tham số	Tỷ lệ phần trăm
C1: Conv $3 \rightarrow 6$, kernel 5×5	456	0.74%
C3: Conv $6 \rightarrow 16$, kernel 5×5	2,416	3.90%
F5: Linear $400 \rightarrow 120$	48,120	77.61%
F6: Linear $120 \rightarrow 84$	10,164	16.39%
Output: Linear $84 \rightarrow 10$	850	1.37%
Tổng cộng	62,006	100.0%

Con số then chốt: **Toàn bộ khối trích xuất đặc trưng tích chập (C1 + C3) chỉ sở hữu 2,872 tham số** — tương ứng vốn vụn 4.63% dung lượng toàn mạng. Sáu bộ lọc ở C1 và mười sáu bộ lọc ở C3 cấu thành toàn bộ vốn biểu diễn không gian mà mô hình sở hữu để bao quát 10 lớp vật thể với sự biến thiên phức tạp về góc chiếu, ánh sáng và bối cảnh.

Đối chiếu: Mạng `SmallCNN` trong `03_cifar10.ipynb` sở hữu **545,098 tham số** và sử dụng cấu hình 32 rồi 64 kênh lọc. LeNet-5 tinh gọn hơn tới **8.79 lần**.

2. Định nghĩa Phiên bản Hiện đại hoá (Modernized)

Thành phần	LeNet-5 nguyên bản (1998)	Phiên bản Hiện đại hoá (Thực nghiệm)
Hàm phi tuyến	<code>tanh</code>	ReLU
Thao tác Pooling	Average Pooling	Max Pooling
Cấu trúc Topo	6→16 kênh, kernel 5×5, head 120→84→10	Bảo tồn chuẩn xác
Thuật toán tối ưu	SGD thuần túy	Adam (chuẩn hoá chung)

Sự chuẩn hóa này là yêu cầu kỹ thuật tất yếu: Thư viện nền tảng `scratch.nn.py` tập trung hỗ trợ `ReLU` và `MaxPool2D`. Sự thống nhất này cho phép cả ba framework thực thi chính xác cùng một đồ thị tính toán mà không làm biến dạng hành vi của mạng.

3. Quy chuẩn Thử nghiệm

Giữ cố định: Cùng tập dữ liệu + Cùng phép phân chia train/test + Cùng siêu tham số. Biến số duy nhất là kiến trúc mạng.

Tham số cấu hình	Giá trị thiết lập
Tập con so sánh 3 framework	5,000 mẫu train / 2,000 mẫu test
Tập chuẩn tham chiếu toàn phần	50,000 mẫu train / 10,000 mẫu test
Số epoch huấn luyện	5 epochs
Kích thước Batch	64
Thuật toán tối ưu	Adam, Learning Rate = 1e-3
Random Seed	42 (tái lập đồng bộ hạt giống ngẫu nhiên)
Chuẩn hoá ảnh	Chuẩn hoá từng kênh dựa trên tập huấn luyện

Chỉ số thời gian huấn luyện `train_seconds` giữa PyTorch (sử dụng GPU RTX 3060 CUDA) và Keras (sử dụng CPU) tiếp tục thể hiện sự chênh lệch môi trường phần cứng, trong khi độ chính xác và dung lượng tham số là hoàn toàn tương đương và công bằng.

4. Kết quả Thực nghiệm Định lượng

4.1 So sánh Ba Framework trên Tập con 5k

Nền tảng thực thi	Số tham số	Accuracy	Macro-F1	Thời gian (s)	Phần cứng
PyTorch	62,006	45.10%	0.4474	1.48	GPU (RTX 3060)
TensorFlow/Keras	62,006	44.70%	0.4418	2.98	CPU
Scratch (NumPy)	62,006	41.40%	0.4079	550.20	CPU

Cả ba nền tảng một lần nữa khẳng định tính nhất quán tuyệt đối ở con số **62,006 tham số**. Khoảng cách độ chính xác giữa ba bản đạt 3.70 điểm phần trăm (41.40% đến 45.10%), rộng hơn rõ rệt so với mức 0.35% trên MNIST. Khi giải quyết một bài toán phức tạp trên cỡ mẫu nhỏ (5,000 ảnh), mô hình trở nên nhạy cảm hơn nhiều với thứ tự xáo trộn mini-batch và phân phối khởi tạo ban đầu.

4.2 Đánh giá trên Toàn bộ Tập dữ liệu 50k

Nền tảng	Accuracy	Macro-F1	Thời gian chạy (s)
PyTorch	61.18%	0.6143	11.25
TensorFlow/Keras	61.11%	0.6074	15.84

Khi mở rộng quy mô dữ liệu từ 5k lên đủ 50k ảnh, độ chính xác tăng vọt **hơn 16 điểm phần trăm** (từ 45.10% lên 61.18%), khẳng định không gian đặc trưng của CIFAR-10 còn cách rất xa ngưỡng bão hòa.

5. Đối đầu Thực nghiệm: LeNet-5 vs SmallCNN

Kiến trúc `SmallCNN` (Notebook 03) gồm 2 tầng conv 3×3 với 32 và 64 kênh, dẫn vào `Linear(4096 → 128) → Linear(128 → 10)`. Tổng số tham số đạt **545,098 — gấp 8.79 lần LeNet-5**.

5.1 Bảng Tổng hợp Đối đầu

Quy mô dữ liệu	Khung làm việc	SmallCNN (545k params)	LeNet-5 (62k params)	Khoảng cách tụt hậu
CIFAR-10 (Subset 5k)	Scratch (NumPy)	51.20%	41.40%	−9.80%
CIFAR-10 (Subset 5k)	TensorFlow/Keras	54.50%	44.70%	−9.80%
CIFAR-10 (Subset 5k)	PyTorch	52.90%	45.10%	−7.80%
CIFAR-10 (Toàn phần 50k)	TensorFlow/Keras	70.11%	61.11%	−9.00%
CIFAR-10 (Toàn phần 50k)	PyTorch	71.84%	61.18%	−10.66%

Khoảng cách thua sút trung bình: −9.41 điểm phần trăm. LeNet-5 bị áp đảo toàn diện ở cả 5 kịch bản thử nghiệm. Kết quả này đối lập sâu sắc với MNIST, nơi LeNet-5 hòa điểm với baseline.

5.2 Hiện tượng Nghẽn Dung lượng Biểu diễn (Capacity Bottleneck)

Khoảng cách thất thế trung bình giữa hai mô hình: - Trên tập con 5k: **−9.13 điểm phần trăm**
- Trên tập toàn phần 50k: **−9.83 điểm phần trăm**

Thông thường, trực giác cho rằng bổ sung dữ liệu sẽ giúp thu hẹp khoảng cách giữa các mô hình. Tuy nhiên trong thí nghiệm này, **khoảng cách tụt hậu lại bị nới rộng thêm khi có thêm dữ liệu huấn luyện**:

LeNet-5 (PyTorch):	45.10%	→	61.18%	(+16.08 điểm)
SmallCNN (PyTorch):	52.90%	→	71.84%	(+18.94 điểm)

Đây là biểu hiện mẫu mực của hiện tượng **Nghẽn dung lượng kiến trúc (Capacity Bottleneck)**: Khi một mạng nơ-ron không đủ dung lượng tham số để hấp thụ và biểu diễn các biến thiên phức tạp trong dữ liệu, việc bổ sung thêm mẫu huấn luyện chỉ mang lại hiệu quả đến một giới hạn nhất định. Mô hình đơn giản là **không còn không gian lưu trữ để mã hóa thêm các đặc trưng mới**.

Nguyên lý kiến trúc: Dữ liệu dồi dào không thể cứu vãn một kiến trúc thiếu hụt dung lượng trầm trọng. Hai nhân tố này bổ trợ nhưng không thể thay thế lẫn nhau.

5.3 Điểm nghẽn nằm ở đâu? Khối Tích chập hay Khối Phân loại?

Mô hình	Tham số Khối Trích xuất Conv	Tham số Khối Phân loại Dense	Tổng tham số
LeNet-5	2,872 (4.6%)	59,134	62,006
SmallCNN	19,392 (3.6%)	525,706	545,098

LeNet-5 chỉ có **6 rồi 16 bộ lọc**. SmallCNN trang bị **32 rồi 64 bộ lọc**. Với chữ số viết tay đơn sắc, 6 bộ lọc là đủ để nắm bắt các nét thẳng, nét cong và góc nhọn. Song với ảnh chụp màu về tàu thủy, máy bay và sinh vật trong môi trường tự nhiên, 6 bộ lọc ở C1 **hoàn toàn cạn kiệt vốn biểu diễn thị giác**. Tất cả các tầng phía sau buộc phải xử lý dựa trên nguồn thông tin nghèo nàn mà C1 giữ lại. Điểm nghẽn nghiêm trọng xuất hiện ngay từ ngưỡng cửa vào của mô hình.

5.4 Đánh đổi Tính toán vs Hiệu năng

Kiến trúc	Tổng tham số	Số phép tính Conv MACs	Thời gian chạy Scratch CPU (s)
SmallCNN	545,098	5,603,328	1,495.13
LeNet-5	62,006	592,800	550.20

LeNet-5 cắt giảm tới **9.45 lần khối lượng tính toán tích chập** và vận hành nhanh hơn 2.72 lần trên CPU NumPy. Nếu mục tiêu độc nhất là tối ưu hóa tài nguyên phần cứng nhúng cực hạn, đây là ưu thế đáng kể. Tuy nhiên dưới góc độ nhận diện chính xác, việc đánh đổi gần 10 điểm phần trăm độ chính xác lấy sự tinh giản này là **một sự đánh đổi bất lợi trên CIFAR-10**.

5.5 Phân tích Ma trận Nhầm lẫn (Confusion Matrix)

Dữ liệu Recall theo từng lớp trên lần chạy PyTorch trên tập 50k (61.18%):

Lớp đối tượng	Tỷ lệ Recall	Lớp đối tượng	Tỷ lệ Recall
ship (Tàu thủy)	0.768	horse (Ngựa)	0.608
automobile (Ô tô)	0.697	cat (Mèo)	0.559
frog (Ếch)	0.690	deer (Hươu)	0.544
truck (Xe tải)	0.677	bird (Chim)	0.496
airplane (Máy bay)	0.643	dog (Chó)	0.436

Cặp nhầm lẫn nghiêm trọng nhất: **Chó → Mèo (ghi nhận 306 trường hợp bị phân loại sai)**.

Mô hình phân loại tương đối chuẩn xác các lớp có **đường nét hình học sắc cạnh hoặc bối cảnh môi trường đặc thù**: tàu thủy (nền biển bao quanh), ô tô / xe tải (đường nét cơ khí,

bánh xe), ếch (màu xanh đặc trưng). Ngược lại, mô hình gặp khó khăn rõ rệt ở các nhóm **động vật có vú bốn chân** (chó, mèo, hươu) — nơi ranh giới phân lớp đòi hỏi phân biệt các chi tiết vi mô như lông thú, hình thái tai hay mõm. Sáu bộ lọc ban đầu không thể trích xuất nổi các thông tin tinh tế này.

6. Đúc kết Toàn diện

1. **LeNet-5 bộc lộ giới hạn năng lực rõ rệt trên CIFAR-10**: Thua sút trung bình 9.41 điểm ở mọi tình huống, và khoảng cách bị kéo giãn rộng hơn khi nâng lượng dữ liệu.
2. **Kích thước hình học không quyết định độ phức tạp dữ liệu**: Dù cùng độ phân giải 32×32 , sự phân hóa nội dung phong phú của ảnh tự nhiên đòi hỏi số kênh trích xuất lớn hơn gấp nhiều lần so với ảnh chữ số nhị phân.
3. **Bài học thiết kế mạng nơ-ron**: Tuyệt đối tránh để xuất hiện điểm nghẽn dung lượng ngay từ tầng tích chập ban đầu.

Mối liên hệ với Chuỗi Thí nghiệm M1 → M4

Notebook `04_compare.ipynb` khảo sát nâng cấp mô hình theo trục **Cơ chế** (BatchNorm, Residual, Attention) trên một ngân sách tham số cố định. Báo cáo này khảo sát mô hình theo trục **Dung lượng kênh trích xuất**. Khi một mô hình bị nghẽn dung lượng cấu trúc ngay từ đầu như LeNet-5 trên CIFAR-10, việc tích hợp thêm các cơ chế hỗ trợ cũng không thể bù đắp nỗi sự thiếu hụt kênh trích xuất gốc.